

OCR-Aware Question Solving Agent

Take-home task brief

The problem

We have a set of multiple-choice question blocks OCR'd out of scanned exam documents. Each block ships as an image (the original scan crop) paired with its OCR text. The OCR text is not always reliable — the source scans and PDFs are frequently low-resolution or noisy, so numbers, symbols, and option labels can come out garbled or wrong.

Build an agent that solves a question block, and treats a failure to match any of the given options as a signal that the OCR text itself may be broken — not just that the question is hard. When that happens, the agent should revise the question text and try again.

Retry & refinement loop

The agent's core loop, per question block:

1. Solve the question using the image and the OCR text as given.
2. Check the answer against the provided options.
3. If it matches one of the options, return it — done.
4. If it matches none of the options, treat that as evidence the OCR text has an error. Re-read the image, look for likely misreads (digits, symbols, garbled words, mislabeled options), and produce a corrected version of the question text.
5. Re-solve using the corrected text and repeat from step 2.
6. Stop when an answer matches an option, or when a retry cap is hit. You decide the cap and justify it in your write-up.

On success, the final output must tag the result **changed** or **unchanged**, and report the final (possibly corrected) question text alongside the original OCR text. If the retry cap is hit with no match, return your best-guess answer and flag it as unresolved — state how you pick a best guess in that case.

OCR tooling: use either the Nanonets OCR API or the Datalab OCR API — both offer \$15 of free credit, which should comfortably cover this task. Pick whichever fits your pipeline; note your choice in the README.

Synthetic OCR noise for testing

To exercise the retry & refinement loop even on OCR text that already came out clean, write a small function that injects a little realistic noise into the OCR text before it reaches the agent — swapping easily-confused characters, e.g. Persian digits like ۲ for ۳, or letters like پ for ب, and similar look-alike substitutions.

Run some of your test question blocks through this function first, so your test set includes both genuinely noisy OCR and OCR you've deliberately corrupted. Mention in your write-up how many characters or words you perturb per block.

Evaluation criteria

Criterion	What we're looking for
Correctness	Final answers match ground truth across a mix of clean and OCR-noisy blocks.
OCR-error detection	Correctly flags blocks where the text really was corrupted, without over-correcting text that was already fine.
Correction quality	Edits are minimal and plausible fixes to the OCR, not a rewrite of the question.
Retry discipline	The loop respects its declared cap and terminates cleanly.
Output clarity	Each result includes the answer, final question text, changed/unchanged tag, and the original OCR text when changed.
Code quality	Clear structure, sensible error handling, reasonable use of the OCR API.

Deliverables & submission format

- Source code (repo link or zip).
- A README covering setup, how to run it, and which OCR provider you used.
- A short write-up: your retry cap and why, and how you handle the unresolved case.
- Output for the sample question blocks provided, one JSON object per block:

```
{  
  "answer": "C",  
  "question_text": "...",  
  "changed": true,  
  "original_ocr_text": "..."  
}
```

Timeline & deadline

Deadline: 1405-06-14 Saturday

Questions before then go to your Datic contact.